

Reviewer Pack — Minimal Order of Local Systems in Algebraic Topology vs. Fre...

Entry 30db37fc52cc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 16:16:06 UTC

Minimal Order of Local Systems in Algebraic Topology vs. Frege Proof Length

Entry ID: 30db37fc52cc **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 16:16:06 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (specifically, local systems) **Field B** (complexity object): Frege Proof Complexity

Statement:

For any given Boolean formula φ with n variables, the length of a Frege proof for φ is linearly correlated with the minimal order of a locally constant sheaf on a suitable space corresponding to φ , such that the length of the Frege proof is $\Theta(\mu_{\text{loc}}(\text{const_sheaf}))$, where $\mu_{\text{loc}}(\text{const_sheaf})$ is the minimal order of the local system associated with the constant sheaf.

Rationale (proposer's reasoning):

The algebraic topology of locally constant sheaves can capture the complexity of logical structures, and its minimal order may provide a measure for the difficulty of proving certain logical statements. The conjecture suggests that this topological invariant could be used to analyze the complexity of Frege proofs, which are a fundamental construct in proof complexity theory.

Taxonomy category: LocalSystemsToProofComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7ddcf141a14be4c5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all Boolean formulas φ with n variables ($n \leq 40$), the correlation coefficient between the length of a Frege proof and the minimal order of the local system associated with the constant sheaf is greater than or equal to 0.7, using at least 100 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -algebraic topology AND local systems AND Frege proof complexity - Frege proof length IN algebraic topology AND locally constant sheaves - minimal order of local systems IN algebraic topology AND correlation with Frege proof length

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_formula(n):
        if n == 1:
            return 'P'
        op = random.choice(['^', 'v'])
        left = generate_boolean_formula(n // 2)
        right = generate_boolean_formula(n - n // 2 - 1)
        return f'({left} {op} {right})'

    def dpll_solve(phi, assignment):
        if not phi:
            return True
        if not assignment:
            return False

        var = next(iter(assignment))
        for val in [True, False]:
            new_assignment = assignment.copy()
            new_assignment[var] = val
            if dpll_solve(phi, new_assignment):
                return True
        return False

    def frege_proof_length(phi):
```

```

stack = []
for char in phi:
    if char == '(':
        stack.append(char)
    elif char == ')':
        while stack[-1] != '(':
            stack.pop()
        stack.pop()
        stack.append('^')
    else:
        stack.append(char)
return len(stack)

def local_system_order(n):
    # Simplified model for demonstration purposes
    return n

n = random.randint(5, 40)
phi = generate_boolean_formula(n)
proof_length = frege_proof_length(phi)
order = local_system_order(n)

return {
    "metric_name": "Frege Proof Length vs. Local System Order",
    "metric_value": proof_length,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": proof_length == order,
    "counterexample": "" if proof_length == order else f"phi={phi}, proof_length={proof_length}"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='<not applicable>' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
p_sandbox/test_38884ed6.py", line 62, in run_trial
    phi = generate_boolean_formula(n)
          ^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38884ed6.py", line 25, in generate_boolean_formula
    left = generate_boolean_formula(n // 2)
          ^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38884ed6.py", line 25, in generate_boolean_formula
    left = generate_boolean_formula(n // 2)
          ^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38884ed6.py", line 25, in generate_boolean_formula
    left = generate_boolean_formula(n // 2)
          ^^^^^^^^^^^^^^^^^^^^^^^^^
[Previous line repeated 1 more time]
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38884ed6.py", line 26, in generate_boolean_formula
    right = generate_boolean_formula(n - n // 2 - 1)
          ^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38884ed6.py", line 25, in generate_boolean_formula
    left = generate_boolean_formula(n // 2)
          ^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38884ed6.py", line 25, in generate_boolean_formula
    left = generate_boolean_formula(n // 2)
          ^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38884ed6.py", line 25, in generate_boolean_formula
    left = generate_boolean_formula(n // 2)
          ^^^^^^^^^^^^^^^^^^^^^^^^^
[Previous line repeated 988 more times]
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38884ed6.py", line 24, in generate_boolean_formula
    op = random.choice(['^', 'v'])
          ^^^^^^^^^^^^^^^^^^^^^
File "/usr/lib/python3.12/random.py", line 348, in choice
    return seq[self._randbelow(len(seq))]
           ^^^^^^^^^^^^^^^^^^^^^
RecursionError: maximum recursion depth exceeded
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its intended task of calculating the correlation coefficients for the conjecture. | next: Investigate and fix the recursion error in the test code to ensure it can run to completion. Once fixed, re-run the test with at least 100 random seeds to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	66755
2	preregistration	ollama_remote	glm4:latest	0	0	15799
3	novelty	ollama_remote	glm4:latest	0	0	8389
4	novelty	ollama_remote	glm4:latest	0	0	8870
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14292
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14509
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7009
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8433
9	judge	ollama_remote	glm4:latest	0	0	10486

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 154542 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/30db37fc52cc.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/30db37fc52cc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/30db37fc52cc.tar.gz` (if generated)